**Lab 5: MPI Matrix–Vector Multiplication and Benchmarking**

**Course:** CS4745 / CS6025 – Parallel and Distributed Processing
**Lab Duration:** 120 minutes
**Submission:** End of lab session

**Learning Objectives**

By completing this lab, you will be able to:

- Initialize and finalize an MPI program correctly.
- Use blocking **MPI collective communication** routines:
    - MPI_Scatter
    - MPI_Bcast
    - MPI_Gather
- Implement **row-wise data decomposition** for regular workloads.
- Measure execution time using MPI_Wtime().
- Observe how runtime changes when varying:
    - the number of processes
    - the matrix size.
- Understand the **memory requirements of dense matrix computations** in parallel programs.

**Allowed and Disallowed MPI Calls**

**Allowed Communication Routines**

You may use only the following MPI collective operations:

- MPI_Scatter
- MPI_Bcast
- MPI_Gather

**Allowed Utility Routines**

- MPI_Init
- MPI_Finalize
- MPI_Comm_rank
- MPI_Comm_size
- MPI_Wtime

**Not Allowed**

The following are **not permitted in this lab**:

- Point-to-point communication
    - MPI_Send
    - MPI_Recv
    - MPI_Sendrecv
- Non-blocking calls
    - MPI_Isend
    - MPI_Irecv
    - MPI_Iscatter
    - etc.

The goal of this lab is to practice **collective communication patterns**.

**Program Requirements**

Your program must:

1. Be written in **C**.
2. Compile using:

```
mpicc -O3 lab5.c -o lab5
```

3. Accept **one command-line argument**:

```
N
```

Where N is the dimension of:

- matrix **A** of size N × N
- vector **x** of size N

4. Assume:

N is divisible by the number of processes P

5. Finalize MPI cleanly using MPI_Finalize().

**Important: Memory Considerations**

This program uses a **dense matrix (a matrix of which most elements are non-zero)**.

Assume that the dense matrix requires:

memory ≈ $8 \times N^2$ bytes

because a double uses **8 bytes**.

Example memory requirements:

| N | Memory for matrix A |
|---|---|
| 1000 | ~8 MB |
| 5000 | ~200 MB |
| 10000 | ~800 MB |
| 20000 | ~3.2 GB |

Because **rank 0 stores the entire matrix**, extremely large values of N may cause the program to crash with errors such as:

Segmentation fault

or

Out of memory

Do **NOT attempt extremely large values such as N = 1,000,000**.
That would require **8 TB of memory**, which exceeds typical systems.

**Build and Run Instructions**

**Compilation**

mpicc -O3 lab5.c -o lab5

**Execution (Local Machine)**

Example:

mpirun -np 4 ./lab5 1000

Always test locally before running larger experiments.

**Part A: Matrix-Vector Multiplication Implementation**

The goal is to compute:

$y = A \times x$

Where:

- A is an N × N matrix
- x is a vector of size N
- y is a vector of size N

**A1. Initialization and Allocation**

**Step 1**

Rank 0 should read N from the command line.

**Step 2**

Compute:

local_rows = N / number_of_processes

Each process will compute **local_rows rows of the matrix**.

**Step 3**

Memory allocation:

**Rank 0**

Allocate:

A : N × N matrix

y : size N

x : size N

Initialize:

A[i] = 1.0

x[i] = 1.0

**All Processes**

Allocate:

local_A : local_rows × N

local_y : local_rows

x : N

**A2. Data Distribution**

Distribute the matrix rows.

Use:

MPI_Scatter

to send chunks of matrix A from rank 0 to all processes.

Each process receives:

local_rows × N elements

**A3. Broadcast Input Vector**

Broadcast the full vector x from rank 0 to all processes.

Use:

MPI_Bcast

After this step every process has the full vector x.

**A4. Local Computation**

Each process computes its portion of the output vector:

local_y = local_A × x

Use standard nested loops:

```
for each local row i
   for each column j
      local_y[i] += local_A[i*N + j] * x[j]
```

**A5. Gather Results**

Collect the partial results back to rank 0.

Use:

MPI_Gather

Each process sends:

local_rows values

Rank 0 receives the complete vector: y

**Part B: Benchmarking and Speedup**

**B1. Timing**

Measure runtime using:

MPI_Wtime()

Recommended placement:

```
MPI_Barrier()
start_time = MPI_Wtime()


communication + computation


MPI_Barrier()
```

end_time = MPI_Wtime()

Only **rank 0** should print the result.

Example output:

N = 1000, Processes = 4, Time = 0.012 seconds

**B2. Performance Testing**

Run experiments using the following parameters.

**Matrix Sizes**

1000

5000

10000

20000

**Number of Processes**

1

2

4

8

Record the execution time for each configuration.

**B3. Speedup Calculation**

Compute speedup using:

S = T_serial / T_parallel

Where:

T_serial = runtime with P = 1

Example table:

| **N** | **P=1** | **P=2** | **P=4** | **P=8** |
|---|---|---|---|---|
| 1000 | | | | |
| 5000 | | | | |
| 10000 | | | | |

---

**B4. Questions for Discussion**

Answer the following questions in your report:

1. Does the program achieve **linear speedup**?
2. If not, what factors limit scalability?

Consider:

- MPI communication overhead
- Memory bandwidth
- Synchronization costs
- Load imbalance

3. Why does increasing N sometimes improve speedup?
4. Why can extremely large values of N cause crashes or segmentation faults?

Hint: consider the **memory required for a dense matrix**.

**Starter Code**

Complete the following program by implementing the MPI communication steps.

```
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char** argv) {

  MPI_Init(&argc, &argv);

  int rank, size;
  MPI_Comm_rank(MPI_COMM_WORLD, &rank);
  MPI_Comm_size(MPI_COMM_WORLD, &size);

  if (argc < 2) {
    if (rank == 0)
      printf("Usage: %s <matrix_size_N>\n", argv[0]);
```

```
    MPI_Finalize();
    return 0;
}

int N = atoi(argv[1]);
int local_rows = N / size;

double *local_A = (double*) malloc(local_rows * N * sizeof(double));
double *local_y = (double*) malloc(local_rows * sizeof(double));
double *x = (double*) malloc(N * sizeof(double));

double *A = NULL;
double *y = NULL;

if (rank == 0) {

    A = (double*) malloc(N * N * sizeof(double));
    y = (double*) malloc(N * sizeof(double));

    for (int i = 0; i < N*N; i++)
        A[i] = 1.0;

    for (int i = 0; i < N; i++)
        x[i] = 1.0;
}

MPI_Barrier(MPI_COMM_WORLD);
double start_time = MPI_Wtime();
```

```
/* TODO
  1. Scatter matrix A to local_A
*/

/* TODO
  2. Broadcast vector x
*/

/* TODO
  3. Compute local matrix-vector product
*/

/* TODO
  4. Gather results into y
*/

MPI_Barrier(MPI_COMM_WORLD);
double end_time = MPI_Wtime();

if (rank == 0) {
   printf("N = %d, Processes = %d, Time = %f seconds\n",
        N, size, end_time - start_time);
   free(A);
   free(y);
}

free(local_A);
free(local_y);
free(x);
```

```
    MPI_Finalize();
    return 0;
}
```